Docker — Interview Ready Answers

Comprehensive answers with examples for all 13 Docker questions

13 Questions • Dockerfile • Multi-stage Builds • Security Scanning • Image Signing • Volumes • Registries • Caching • Secrets

---

Q1. Write a Dockerfile for a Node.js application with multi-stage builds.

Answer

Multi-stage builds are a Docker best practice for Node.js applications. They separate the build environment (which needs build tools, dev dependencies, and compilers) from the production runtime image (which needs only the compiled application and production dependencies). The result is a dramatically smaller, more secure image.

Why Multi-Stage Builds Matter for Node.js

- A naive Node.js image that includes node_modules from npm install can easily exceed 1GB — including thousands of devDependencies, source maps, test files, and build tools never needed at runtime.
- A multi-stage build produces an image of 150-300MB by copying only the compiled output and production node_modules.
- The build stage never ships to production — it stays on the build machine. Any secrets, source code, or intermediate files used during the build are discarded.

Complete Production-Grade Dockerfile

```
# ─────────────────────────────────────────────────
# Stage 1: Dependencies
# Install ALL dependencies (including devDeps)
# ─────────────────────────────────────────────────
FROM node:20.11.0-alpine3.19 AS deps

# Set working directory
WORKDIR /app

# Copy only package files first (layer caching optimisation)
# This layer is rebuilt ONLY when package.json or lockfile changes
COPY package.json package-lock.json ./

# Install all dependencies including devDependencies
RUN npm ci --frozen-lockfile

# ─────────────────────────────────────────────────
# Stage 2: Builder
# Compile TypeScript, build assets, run tests
# ─────────────────────────────────────────────────
FROM deps AS builder

# Copy source code
COPY . .

# Build the application (TypeScript → JavaScript)
RUN npm run build

# Run tests in the build stage (fails build if tests fail)
# RUN npm run test  # Optional: enable for test-in-build

# Prune devDependencies — only keep production deps
RUN npm prune --production

# ─────────────────────────────────────────────────
# Stage 3: Production Runtime
# Minimal image with only what's needed to run
# ─────────────────────────────────────────────────
FROM node:20.11.0-alpine3.19 AS production

# Security: install security updates
RUN apk update && apk upgrade --no-cache

# Security: create non-root user
```

```
RUN addgroup -g 1001 -S appgroup && \
    adduser  -u 1001 -S appuser -G appgroup

WORKDIR /app

# Copy only production artifacts from builder stage
COPY --from=builder --chown=appuser:appgroup /app/dist ./dist
COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules
COPY --from=builder --chown=appuser:appgroup /app/package.json ./

# Switch to non-root user
USER appuser

# Document the port the application listens on
EXPOSE 3000

# Health check — Docker and Kubernetes can use this
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health', (r) =>
process.exit(r.statusCode === 200 ? 0 : 1))"

# Use exec form (not shell form) to handle signals correctly
ENTRYPOINT ["node"]
CMD ["dist/server.js"]
```

Building the Image

```
# Build with BuildKit enabled (recommended)
DOCKER_BUILDKIT=1 docker build \
  --target production \
  --tag myapp:v1.2.3 \
  --tag myapp:latest \
  .

# Check final image size
docker images myapp
# REPOSITORY  TAG     SIZE
# myapp       v1.2.3  165MB   ← Much smaller than 1GB+ naive build
```

Layer Caching Strategy Explained

The order of COPY instructions is critical for build performance. Docker caches each layer — a layer is only rebuilt when its inputs change:

Layer	Instruction
1	FROM node:20-alpine
2	COPY package*.json ./
3	RUN npm ci
4	COPY . .
5	RUN npm run build

By copying package*.json before the source code, npm ci only reruns when dependencies change — not on every code change. This saves 30-120 seconds per build in typical projects.

Using .dockerignore

Always create a .dockerignore file to prevent unnecessary files from entering the build context, which slows the COPY step and wastes cache:

```
# .dockerignore
node_modules/      # Avoid copying local node_modules — use npm ci inside
.git/              # Never include git history
*.md               # Documentation
*.log              # Log files
.env               # Environment files — never in image
dist/              # Previous build artifacts
coverage/          # Test coverage reports
.github/           # CI/CD config
tests/             # Test files not needed in prod
Dockerfile*        # The Dockerfile itself
```

> *Tip: Pin the Node.js base image to an exact version AND digest for reproducible builds: FROM node:20.11.0-alpine3.19@sha256:abc123… This prevents the image from silently changing when node:20-alpine is updated, which could introduce unexpected behaviour or vulnerabilities.*

Q2. How do you debug a container that has exited?

Answer

When a container exits (whether with an error or unexpectedly), you lose the ability to exec into it — the container is no longer running. Several techniques let you investigate what went wrong without restarting the container and potentially losing the failure state.

Step 1 — Find the Exited Container

```
# List ALL containers including stopped/exited
docker ps -a
# CONTAINER ID  IMAGE    COMMAND   STATUS
# a1b2c3d4e5f6  myapp:v1 'node ..' Exited (1) 5 minutes ago

# Filter to show only exited containers
docker ps -a --filter status=exited

# Kubernetes: find recently crashed pods
kubectl get pods --all-namespaces | grep -v Running
```

Step 2 — Read the Last Logs

```
# Docker — logs from the exited container
docker logs <container-id>
docker logs <container-id> --tail 100   # Last 100 lines
docker logs <container-id> --timestamps # With timestamps

# Kubernetes — logs from the PREVIOUS crashed container
kubectl logs <pod-name> --previous
kubectl logs <pod-name> --previous --tail=200

# Multi-container pod — specify container
kubectl logs <pod-name> -c <container-name> --previous
```

Step 3 — Inspect the Exit Code

```
# Docker — get exit code and other state details
docker inspect <container-id>
docker inspect <container-id> | grep -A5 '"State"'
# "ExitCode": 137   # 128+9 = SIGKILL = OOMKilled
# "ExitCode": 1     # Application error
# "ExitCode": 0     # Exited cleanly (process finished normally)
# "OOMKilled": true # Out of memory

# Kubernetes
kubectl describe pod <pod-name>
# Last State: Terminated
#   Reason: OOMKilled   Exit Code: 137
```

Exit Code	Meaning
0	Clean exit
1	Generic error
137	SIGKILL (128+9)
139	SIGSEGV (128+11)
143	SIGTERM (128+15)
126	Permission denied
127	Command not found
255	Exit(-1)

Step 4 — Run the Image with an Overridden Command

The most powerful debugging technique: start the exited image with a shell instead of the normal ENTRYPOINT, so you can investigate the container's filesystem and environment interactively:

```
# Override ENTRYPOINT to get a shell
docker run --rm -it \
  --entrypoint /bin/sh \
  myapp:v1 \
  -c 'ls -la /app && cat /etc/hosts && env'

# Interactive shell — explore as if the container is running
docker run --rm -it \
  --entrypoint /bin/sh \
  myapp:v1
  # Now explore: ls, cat config files, run the app manually
  # /app $ node dist/server.js   # Run and see the error directly
```

Step 5 — Commit Exited Container and Inspect

If the container modified the filesystem before exiting (e.g., wrote logs or config), you can commit its state to a new image and inspect it:

```
# Commit the exited container's filesystem to an image
docker commit <container-id> debug-snapshot

# Inspect the snapshot
docker run --rm -it debug-snapshot /bin/sh
# Check: what files were created/modified before the crash?
ls -la /app/logs/
cat /app/logs/error.log
```

Step 6 — Check Resource Constraints

```
# Check if container was OOM killed
docker stats --no-stream <container-id>

# Check Docker daemon events for OOM
docker events --filter container=<container-id>

# Linux kernel OOM log
dmesg | grep -i 'oom\|killed'
```

Step 7 — Use docker debug (Docker Desktop)

Docker Desktop 4.29+ includes docker debug, which attaches a debugging shell to a running container even if it has no shell installed — or to examine a stopped container's filesystem:

```
# Attach debug shell to a stopped container
docker debug <container-id>
# Works even with distroless images that have no shell
```

Preventing Future Debugging Difficulty

- Always configure logging — use --log-driver json-file with max-size and max-file to retain logs after container exit.
- Add a HEALTHCHECK in the Dockerfile — Docker marks the container unhealthy before it crashes, providing an early warning.
- Use a restart policy (--restart=on-failure) in dev to study repeated crash patterns.
- Write structured JSON logs — easier to parse and search in log aggregation systems.

> *Tip: The single most useful command for debugging exited containers is docker logs <container-id> combined with docker inspect <container-id> to get the exit code. These two commands together almost always identify the root cause within 30 seconds.*

Q3. What's the difference between COPY and ADD commands in Dockerfile?

Answer

Both COPY and ADD copy files from the build context into the image, but ADD has additional capabilities that make it less predictable and potentially dangerous. Understanding when to use each is a Dockerfile best practice question.

COPY — Simple, Explicit, Preferred

COPY does exactly one thing: copies files or directories from the build context (or another build stage) into the image filesystem. Nothing more, nothing less. Its behaviour is completely predictable.

```
# COPY syntax
COPY <source> <destination>
```

```
COPY <source> ... <destination>

# Examples
COPY package.json ./
COPY src/ /app/src/
COPY package.json package-lock.json ./

# Multi-stage: copy from a previous stage
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

# Copy with ownership (avoids separate chown layer)
COPY --chown=appuser:appgroup src/ /app/src/
```

ADD — Everything COPY Does Plus More

ADD extends COPY with two additional capabilities: automatic extraction of local tar archives, and downloading files from remote URLs. These features make ADD less predictable:

```
# ADD extracts tar archives automatically
ADD app.tar.gz /opt/app/
# The .tar.gz is extracted in-place — no separate RUN tar command needed

# ADD can fetch from a URL
ADD https://example.com/config.json /app/config.json
# This downloads at build time — problematic for reproducibility
```

Feature Comparison

Feature	COPY
Copy local files	Yes
Copy from another build stage (--from)	Yes
Auto-extract .tar, .tar.gz, .tar.xz	No
Download from URL	No
Layer caching behaviour	Predictable
Recommended use	Always — general file copying

Why COPY is Preferred Over ADD

- Explicit is better than implicit — COPY does exactly what you write. ADD has hidden behaviour that can surprise developers.
- Security risk with URL downloads — ADD from a URL downloads at build time over HTTP/HTTPS. The downloaded file is not verified by default, and the URL might change, be compromised, or become unavailable. Better to use RUN curl --fail --silent with explicit verification.
- No caching for URLs — Docker cannot cache URL-based ADD instructions reliably because the URL content might change. This slows builds.
- Unexpected tar extraction — If you COPY a .tar.gz into the image intending to keep it compressed, COPY does that correctly. ADD would silently extract it.

When ADD is Acceptable

The only legitimate use case for ADD over COPY is when you specifically want automatic tar extraction and the source is a LOCAL tar file (not a URL):

```
# Acceptable: local tar extraction
ADD app-v1.2.tar.gz /opt/app/
# Equivalent but more explicit with COPY:
COPY app-v1.2.tar.gz /tmp/
RUN tar -xzf /tmp/app-v1.2.tar.gz -C /opt/app/ && rm /tmp/app-v1.2.tar.gz
```

What to Use Instead of ADD for URLs

```
# Instead of: ADD https://example.com/file.sh /tmp/
# Use RUN with curl for better control:
RUN curl --fail --silent --location \
    --output /tmp/file.sh \
    https://example.com/file.sh && \
```

```
    echo 'expected-sha256-hash  /tmp/file.sh' | sha256sum -c - && \
    chmod +x /tmp/file.sh
```

This approach explicitly verifies the downloaded file's integrity via SHA256 checksum — a critical security control that ADD cannot provide.

> *Tip: Follow Docker's official best practice: use COPY unless you specifically need tar extraction. The Dockerfile linter hadolint enforces this and raises DL3020 (Use COPY instead of ADD for files and folders) whenever ADD is used unnecessarily.*

Q4. How would you handle secrets in a Docker container for a PHP application connecting to MySQL?

Answer

Secrets in Docker containers is one of the most common security mistakes. Database passwords, API keys, and connection strings must never be hardcoded in images, Dockerfiles, or environment variables that end up in image layers. Multiple safe patterns exist.

What NOT to Do — Common Anti-Patterns

```
# WRONG: Hardcoded in Dockerfile ENV
ENV DB_PASSWORD=mysecretpassword   # Visible in docker inspect and image history

# WRONG: Passed as build ARG
ARG DB_PASSWORD
RUN echo $DB_PASSWORD              # Visible in docker history

# WRONG: In .env file committed to Git
# DB_PASSWORD=mysecretpassword     # In .env in source control
```

Method 1 — Runtime Environment Variables (Development)

Pass secrets at container startup as environment variables. The value is never baked into the image. This is acceptable for development and simple setups:

```
# docker-compose.yml
services:
  php-app:
    image: myapp:latest
    environment:
      DB_HOST: mysql
      DB_NAME: myapp
      DB_USER: appuser
      DB_PASSWORD: ${DB_PASSWORD}   # Reads from shell environment
    env_file:
      - .env.local                  # Local only — in .gitignore

# .env.local (NEVER commit to Git)
DB_PASSWORD=mysecretpassword
DB_USER=appuser

# PHP application reads from environment
$dsn = sprintf('mysql:host=%s;dbname=%s', getenv('DB_HOST'), getenv('DB_NAME'));
$pdo = new PDO($dsn, getenv('DB_USER'), getenv('DB_PASSWORD'));
```

> *Note: Environment variables are visible in docker inspect, process listings (ps aux), and are accessible to all processes in the container. They are acceptable for development but should be replaced with a secrets manager in production.*

Method 2 — Docker Secrets (Docker Swarm / Production)

Docker Secrets store encrypted secret data in the Swarm's distributed store and mount them as files inside containers at /run/secrets/<secret-name>. The secret value is never stored as an environment variable:

```
# Create a Docker secret
echo 'mysecretpassword' | docker secret create db_password -

# docker-compose.yml (Swarm mode)
services:
  php-app:
    image: myapp:latest
    secrets:
      - db_password
      - db_user

secrets:
```

```
  db_password:
    external: true
  db_user:
    external: true

# PHP reads the secret from file (mounted at /run/secrets/)
$db_password = trim(file_get_contents('/run/secrets/db_password'));
$db_user     = trim(file_get_contents('/run/secrets/db_user'));
$pdo = new PDO($dsn, $db_user, $db_password);
```

Method 3 — AWS Secrets Manager / HashiCorp Vault (Cloud Production)

For cloud environments, the application retrieves secrets at startup from a secrets manager using an IAM role (no credentials needed):

```
# PHP using AWS SDK for Secrets Manager
use Aws\SecretsManager\SecretsManagerClient;

$client = new SecretsManagerClient(['region' => 'us-east-1']);
$result = $client->getSecretValue(['SecretId' => 'prod/myapp/db-creds']);
$secret = json_decode($result['SecretString'], true);

$pdo = new PDO(
    'mysql:host=...',
    $secret['username'],
    $secret['password']
);
# Application uses IAM role — no credentials in the container
```

Method 4 — Docker BuildKit Secrets (Build-Time Secrets)

When a secret is needed only during the build (e.g., a private npm registry token or Composer token), BuildKit secrets mount the secret as a temporary file that is NOT stored in any layer:

```
# Dockerfile — using BuildKit secret
# syntax=docker/dockerfile:1
FROM php:8.3-alpine

# Mount secret for composer authentication — not stored in image
RUN --mount=type=secret,id=composer_token \
    COMPOSER_AUTH="{\"github-oauth\":{\"github.com\":\"$(cat
/run/secrets/composer_token)\"}}" \
    composer install --no-dev --optimize-autoloader

# Build with secret (never stored in any layer)
DOCKER_BUILDKIT=1 docker build \
    --secret id=composer_token,src=~/.composer/auth.json \
    -t myapp:latest .
```

Method 5 — Kubernetes Secrets (for EKS/AKS/GKE)

In Kubernetes, secrets are passed to PHP pods as environment variables or mounted files from Kubernetes Secret objects:

```
# Mount DB credentials as files in the pod
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysql-credentials
      key: password
```

Comparison of Methods

Method	Security Level
Hardcoded in image	NONE — avoid completely
Runtime env vars	Low — visible in inspect
Docker Secrets	Medium — encrypted at rest
BuildKit secrets	High — never in layers
AWS/Azure Secrets	High — centrally

Manager	managed, rotatable
Kubernetes Secrets + ESO	High with encryption

> *Tip: Use PHP's getenv() to read secrets at runtime rather than hardcoding connection strings. If moving to cloud, retrieve secrets from AWS Secrets Manager at application startup and cache them in-process. Enable automatic rotation in Secrets Manager and handle the rotation in your PHP application with a retry mechanism.*

Q5. What are the stages in a Docker image build? Why do we use ENTRYPOINT and CMD instructions?

Answer

Understanding Docker's build process and the distinction between ENTRYPOINT and CMD is fundamental to writing correct Dockerfiles — especially when containers need to handle arguments gracefully or integrate with orchestrators.

Stages of a Docker Image Build

Stage 1 — Build Context Preparation

When you run docker build ., Docker reads the entire current directory (or specified path) and sends it to the Docker daemon as the build context. Files in .dockerignore are excluded. Smaller build contexts mean faster builds:

```
docker build -t myapp:latest .
# Sending build context to Docker daemon  2.3MB
# Context size depends on directory content and .dockerignore
```

Stage 2 — Instruction Processing (Layer by Layer)

Docker processes each instruction in the Dockerfile sequentially, creating a new read-only filesystem layer for each instruction that modifies the filesystem (RUN, COPY, ADD). Instructions that only set metadata (ENV, EXPOSE, LABEL, ENTRYPOINT, CMD) do not create separate layers in the union filesystem:

Instruction	Creates a layer?
FROM	Yes (base layer)
RUN	Yes
COPY	Yes
ADD	Yes
ENV	No (metadata)
EXPOSE	No (metadata)
LABEL	No (metadata)
ENTRYPOINT	No (metadata)
CMD	No (metadata)
WORKDIR	Yes (dir creation)
USER	No (metadata)

Stage 3 — Layer Caching

Docker checks its cache before executing each instruction. If the instruction and all preceding instructions are identical AND the input files (for COPY/ADD) haven't changed, Docker reuses the cached layer and skips execution. The first cache miss invalidates all subsequent layers.

Stage 4 — Image Assembly

All layers are stacked using a Union Filesystem (OverlayFS on Linux). The final image is a combination of all layers plus the metadata (ENTRYPOINT, CMD, ENV, EXPOSE, etc.). Docker assigns a SHA256 digest to the final image.

ENTRYPOINT — The Fixed Executable

ENTRYPOINT defines the executable that always runs when the container starts. It cannot be overridden by arguments passed to docker run (only --entrypoint flag overrides it). Think of it as: 'this container IS this process.'

```
# Exec form (preferred) — runs directly, no shell
ENTRYPOINT ["node", "server.js"]

# Shell form (avoid) — runs in /bin/sh -c
ENTRYPOINT node server.js
```

```
# docker run myapp
# Runs: node server.js   (ENTRYPOINT cannot be changed by args)

# docker run myapp --port 8080
# Runs: node server.js --port 8080  (args appended to ENTRYPOINT)
```

CMD — The Default Arguments

CMD provides default arguments to ENTRYPOINT, or the default command if no ENTRYPOINT is set. CMD IS overridden by arguments passed to docker run. Think of it as: 'run this unless told otherwise.'

```
# CMD as default arguments to ENTRYPOINT
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
# docker run nginx-image → runs: nginx -g 'daemon off;'
# docker run nginx-image -g 'daemon on;' → runs: nginx -g 'daemon on;'
  # CMD is overridden; ENTRYPOINT stays

# CMD as standalone command (no ENTRYPOINT)
CMD ["python", "-m", "http.server", "8000"]
# docker run my-image → runs: python -m http.server 8000
# docker run my-image bash → runs: bash  (CMD fully overridden)
```

ENTRYPOINT vs CMD — Decision Matrix

Scenario	Configuration
Fixed app, no configurable args	ENTRYPOINT only
App with configurable defaults	ENTRYPOINT + CMD
General-purpose base image	CMD only
Wrapper script pattern	ENTRYPOINT (script) + CMD (args)

Wrapper Script Pattern (Production Best Practice)

Use an entrypoint.sh script for pre-startup tasks (config generation, database waiting, secret retrieval) and pass CMD as arguments:

```
#!/bin/sh
# entrypoint.sh
set -e

# Wait for database to be ready
until nc -z $DB_HOST 3306; do
  echo 'Waiting for database...'
  sleep 2
done

# Run database migrations
php artisan migrate --force

# Execute the main process (passed as CMD arguments)
exec "$@"

# Dockerfile
ENTRYPOINT ["/entrypoint.sh"]
CMD ["php", "-S", "0.0.0.0:8000", "-t", "public"]
```

The exec "$@" at the end replaces the shell with the CMD process, ensuring proper signal handling (SIGTERM reaches the application, not just the shell).

Shell Form vs Exec Form

Aspect	Shell form: CMD node app.js
Signal handling	SIGTERM goes to /bin/sh, NOT node
PID	node runs as PID 2+
Shell variable expansion	Yes ($ENV_VAR works)
Recommended for	Simple scripts with

	variables

> *Tip: Always use exec form (JSON array) for ENTRYPOINT and CMD in production. Shell form wraps commands in '/bin/sh -c', which means your application process receives signals (SIGTERM for graceful shutdown) through the shell, not directly — causing containers to be killed abruptly rather than shutting down gracefully.*

Q6. Which container registry do you use for storing Docker images?

Answer

Container registries are the distribution layer for Docker images. The right choice depends on your cloud provider, security requirements, performance needs, and team size. Below is a comprehensive overview of registries used in production environments.

1. Amazon ECR (Elastic Container Registry)

The native registry for AWS environments. Integrates seamlessly with EKS, ECS, and Lambda without needing separate credentials — IAM roles authenticate automatically.

- Authentication via IAM — no long-lived credentials; EKS nodes pull images using their instance role.
- Cross-region and cross-account replication — images automatically mirrored to other regions.
- Enhanced scanning — Amazon Inspector continuously scans images for CVEs, even after push.
- Lifecycle policies — automatically delete images older than N days or keep only the last N images.

```
# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS \
  --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

# Push image
docker tag myapp:v1.2 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.2
docker push \
  123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.2

# Create repository with scanning enabled
aws ecr create-repository \
  --repository-name myapp \
  --image-scanning-configuration scanOnPush=true
```

2. Azure Container Registry (ACR)

The native registry for Azure environments. Integrates with AKS using Workload Identity Federation — no credentials needed.

- Geo-replication — single registry replicated to multiple Azure regions automatically.
- Build tasks — ACR can build images directly from source code using ACR Tasks.
- Defender for Containers — vulnerability scanning integrated with Microsoft Defender.

```
# Authenticate to ACR
az acr login --name mycompanyacr

# Push image
docker tag myapp:v1.2 mycompanyacr.azurecr.io/myapp:v1.2
docker push mycompanyacr.azurecr.io/myapp:v1.2

# Grant AKS pull access (Workload Identity)
az aks update \
  --name my-aks --resource-group my-rg \
  --attach-acr mycompanyacr
```

3. GitHub Container Registry (GHCR)

Integrated with GitHub Actions workflows. Images are stored alongside the code that produces them. Free for public repositories; included in GitHub paid plans.

```
# In GitHub Actions — authenticate with GITHUB_TOKEN
- name: Log in to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}
```

```
- name: Push image
  run: |
    docker tag myapp:latest ghcr.io/${{ github.repository }}/myapp:${{ github.sha }}
    docker push ghcr.io/${{ github.repository }}/myapp:${{ github.sha }}
```

4. Docker Hub

The original public registry. Free tier available with rate limits (100 pulls/6 hours for anonymous, 200/6 hours for free accounts). Used for public open-source images and development. Pull rate limits make it unsuitable as a primary registry for production CI/CD.

5. JFrog Artifactory / Nexus (On-Premise / Hybrid)

Enterprise registries that support multiple artifact types (Docker, npm, Maven, PyPI). Used when data residency requirements prevent use of cloud registries, or when a single repository manager for all artifact types is needed.

6. Google Artifact Registry (GAR)

The modern replacement for Google Container Registry (GCR). Native integration with GKE and Cloud Build. Supports Docker, npm, Maven, and Python packages.

Registry Selection Criteria

Registry	Best For
AWS ECR	AWS/EKS environments
Azure ACR	Azure/AKS environments
GHCR	GitHub Actions CI/CD
Docker Hub	Public images, development
JFrog Artifactory	Enterprise, multi-format
Google GAR	GCP/GKE environments

Best Practices for Any Registry

- Use immutable image tags — tag with git commit SHA (abc123) not 'latest'. Immutable tags prevent silent image changes.
- Enable vulnerability scanning on every push — block deployment if CRITICAL CVEs are found.
- Set lifecycle policies — automatically purge images older than 90 days or beyond the last N versions.
- Restrict push access — only CI/CD pipelines should have write access. Developers have pull access only.
- Enable image signing — sign images with cosign (Sigstore) and verify signatures in admission controllers.

> *Tip: For AWS environments, use ECR private repositories with IAM role authentication for EKS. This eliminates the need for imagePullSecrets in Kubernetes — EKS worker nodes authenticate to ECR automatically using their instance profile IAM role.*

Q7. Are you aware of security scanning tools? How do you scan Docker images — both during build and at the registry level? Are you using any extensions or tools for image scanning?

Answer

Container image security scanning identifies known vulnerabilities (CVEs), misconfigurations, exposed secrets, and policy violations in Docker images. Scanning must happen at multiple points: during CI builds (shift-left) and continuously in the registry (for newly discovered vulnerabilities).

Why Scanning at Both Stages is Required

A vulnerability-free image at build time may become vulnerable next week when a new CVE is published for one of its packages. Registry-level continuous scanning catches these post-push vulnerabilities without requiring a new build.

Build-Time Scanning Tools

1. Trivy (Most Popular — Aqua Security)

Trivy is the most widely used open-source container scanner. It scans for OS CVEs, language package vulnerabilities (npm, pip, Maven), misconfigurations, secrets, and Kubernetes manifests.

```
# Install Trivy
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh

# Scan a local image
trivy image myapp:latest
```

```
# Scan with severity filter — fail on CRITICAL or HIGH
trivy image \
  --exit-code 1 \
  --severity CRITICAL,HIGH \
  --no-progress \
  myapp:latest

# Output as SARIF for GitHub Security tab
trivy image \
  --format sarif \
  --output trivy-results.sarif \
  myapp:latest

# Scan IaC and Kubernetes manifests too
trivy config ./kubernetes/
trivy config ./terraform/

# Scan filesystem (before building image)
trivy fs --scanners vuln,secret,misconfig .
```

Trivy in GitHub Actions CI/CD

```
- name: Build Docker image
  run: docker build -t myapp:${{ github.sha }} .

- name: Scan image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: myapp:${{ github.sha }}
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: '1'

- name: Upload results to GitHub Security
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: trivy-results.sarif
```

2. Grype (Anchore)

Grype focuses on CVE detection with a very large vulnerability database. Often used alongside Trivy for comparison or in Anchore Enterprise pipelines.

```
# Install and scan
grype myapp:latest
grype myapp:latest --fail-on critical
```

3. Snyk

Snyk provides both open-source scanning and a SaaS platform with developer-friendly remediation advice. Integrates with GitHub, GitLab, Azure DevOps, and Jenkins.

```
# Scan with Snyk
snyk container test myapp:latest
snyk container monitor myapp:latest   # Send results to Snyk dashboard
```

4. Docker Scout (Built into Docker)

Docker Scout is Docker's built-in vulnerability scanner available in Docker Desktop and Docker Hub. Provides real-time CVE analysis and fix recommendations.

```
# Scan with Docker Scout
docker scout cves myapp:latest
docker scout recommendations myapp:latest
docker scout quickview myapp:latest
```

5. Hadolint — Dockerfile Linting

Hadolint statically analyses Dockerfiles for best practice violations before even building the image. Catches insecure patterns, deprecated instructions, and style issues.

```
hadolint Dockerfile
# DL3007 Using latest is prone to errors
# DL3008 Pin versions in apt-get install
# DL3020 Use COPY instead of ADD
# SC2046 Quote this to prevent word splitting (shell warnings)
```

Registry-Level Continuous Scanning

- AWS ECR with Amazon Inspector — Automatically scans images on push AND monitors continuously for new CVEs published after the push. Alerts via SNS and integrates with Security Hub.
- Azure ACR with Microsoft Defender for Containers — Continuous scanning with integration into Microsoft Defender for Cloud. Provides compliance reporting.
- JFrog Xray — Advanced scanning with policy enforcement, impact analysis, and SBOM generation. Can block downloads of images with violations.
- Snyk Container Registry Integration — Connects to ECR, ACR, Docker Hub, and GCR to continuously monitor pushed images.

Scanning Strategy by Stage

Stage	Tool
Developer local	Docker Scout / Trivy
CI build	Trivy / Grype
Pre-push to registry	Trivy
Registry	ECR Inspector / Defender
Pre-deploy admission	Kyverno / OPA

SBOM Generation

Generate a Software Bill of Materials (SBOM) to document all components in the image — required for supply chain security and compliance frameworks (NIST, SLSA):

```
# Generate SBOM with Trivy
trivy image --format cyclonedx --output sbom.json myapp:latest

# Generate SBOM with Syft
syft myapp:latest -o spdx-json > sbom.spdx.json
```

> *Tip: Start with Trivy in your CI pipeline — it's free, fast, and covers OS packages, language dependencies, secrets, and IaC misconfigurations in one tool. Combine with ECR/ACR continuous scanning in the registry for comprehensive coverage across the image lifecycle.*

Q8. How do you pass environment variables during Docker build commands? What services do you use for storing Docker images?

Answer

Docker provides two mechanisms for passing values during a build: ARG (build-time variables) and ENV (environment variables that persist in the image). Understanding their differences and security implications is critical.

ARG — Build-Time Variables

ARG variables exist only during the build process. They are not present in the final running container (unless explicitly copied to ENV). They can be passed via --build-arg on the command line.

```
# Dockerfile — declare ARG
ARG APP_VERSION=1.0.0          # With default
ARG BUILD_DATE               # Without default — must be passed
ARG NODE_ENV=production

# Use ARG value in build instructions
RUN echo "Building version $APP_VERSION"
LABEL version="$APP_VERSION" build-date="$BUILD_DATE"

# Make ARG available at runtime by copying to ENV
ENV APP_VERSION=$APP_VERSION   # Now available in running container

# Pass values at build time
docker build \
  --build-arg APP_VERSION=2.1.0 \
  --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  -t myapp:2.1.0 .
```

ENV — Runtime Environment Variables

ENV variables are baked into the image and present in all containers started from it. They are the persistent configuration mechanism for containerised applications:

```
# Dockerfile — set ENV
ENV PORT=3000 \
    NODE_ENV=production \
    LOG_LEVEL=warn

# Override at runtime (does not change the image)
docker run -e PORT=8080 -e LOG_LEVEL=debug myapp:latest

# From a file
docker run --env-file .env.production myapp:latest
```

ARG vs ENV Comparison

Aspect	ARG
Available during build	Yes
Available in running container	No (unless copied to ENV)
Visible in docker inspect	Yes — in .Config.Env
Visible in docker history	Yes — security risk!
Use for secrets	NEVER
Override at runtime	No
Best for	Version numbers, build flags

> *Note: Both ARG and ENV values used in RUN instructions are visible in docker history and docker inspect. NEVER use them for secrets (passwords, API keys). Use BuildKit secrets (--mount=type=secret) or retrieve secrets at container startup from a secrets manager.*

BuildKit Secrets — The Secure Alternative for Build-Time Secrets

```
# Dockerfile with BuildKit secret
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./

# Secret mounted as temporary file — NOT stored in any layer
RUN --mount=type=secret,id=npm_token \
    NPM_TOKEN=$(cat /run/secrets/npm_token) \
    npm config set '//registry.npmjs.org/:_authToken' $NPM_TOKEN && \
    npm ci && \
    npm config delete '//registry.npmjs.org/:_authToken'

# Build with BuildKit secret
DOCKER_BUILDKIT=1 docker build \
  --secret id=npm_token,env=NPM_TOKEN \
  -t myapp:latest .

# In GitHub Actions
- uses: docker/build-push-action@v5
  with:
    secrets: |
      npm_token=${{ secrets.NPM_TOKEN }}
```

Passing Build Args in CI/CD

```
# GitHub Actions — pass dynamic build args
- name: Build and push
  uses: docker/build-push-action@v5
  with:
    push: true
    tags: myapp:${{ github.sha }}
    build-args: |
      APP_VERSION=${{ github.ref_name }}
      BUILD_DATE=${{ github.event.head_commit.timestamp }}
      GIT_COMMIT=${{ github.sha }}
```

Services for Storing Docker Images

See Q6 for a comprehensive comparison of container registries. Summary: AWS ECR for AWS environments, Azure ACR for Azure, GHCR for GitHub Actions pipelines, and JFrog Artifactory for enterprise multi-format needs.

> *Tip: Embed build metadata (version, commit SHA, build date) as image labels rather than environment variables. Labels are queryable (docker inspect) without running the container, and they don't affect the running application's environment: LABEL org.opencontainers.image.version='1.2.3' org.opencontainers.image.revision='abc123'.*

Q9. What is image signing and Helm chart signing? Which tools do you use to sign Helm charts?

Answer

Image and chart signing is a supply chain security control that ensures the artifacts deployed to production are exactly what the CI/CD pipeline produced — and haven't been tampered with, substituted, or compromised.

Why Signing Matters

Without signing, anyone who can push to your container registry can push a malicious image with the same tag. An attacker who compromises the registry or a developer's credentials could substitute images silently. Signing creates a cryptographic proof of origin and integrity.

Docker Image Signing — cosign (Sigstore)

cosign is the modern standard for container image signing, part of the Sigstore open-source project (backed by Google, Red Hat, Chainguard). It stores signatures alongside images in the registry as OCI artifacts.

```
# Install cosign
brew install cosign  # or
curl -O -L https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-amd64
chmod +x cosign-linux-amd64 && mv cosign-linux-amd64 /usr/local/bin/cosign

# Method 1: Keyless signing (recommended — uses OIDC from CI)
# In GitHub Actions, cosign uses GITHUB_TOKEN for keyless signing
- name: Sign image with cosign (keyless)
  env:
    COSIGN_EXPERIMENTAL: '1'
  run: |
    cosign sign \
      --yes \
      myacr.io/myapp@${{ steps.build.outputs.digest }}
    # Signature stored in registry alongside image
    # Verified via Sigstore transparency log (Rekor)

# Method 2: Key-based signing
# Generate key pair
cosign generate-key-pair
# Produces: cosign.key (private) and cosign.pub (public)

# Sign
cosign sign --key cosign.key myacr.io/myapp:v1.2

# Verify
cosign verify --key cosign.pub myacr.io/myapp:v1.2
```

Enforcing Signature Verification in Kubernetes

Use Kyverno or OPA Gatekeeper as admission controllers to reject pods that use unsigned images:

```
# Kyverno policy — require cosign signature
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-signed-images
spec:
  rules:
  - name: check-image-signature
    match:
      resources: { kinds: [Pod] }
    verifyImages:
    - imageReferences: ['myacr.io/*']
      attestors:
      - entries:
        - keys:
            publicKeys: |-
```

```
              -----BEGIN PUBLIC KEY-----
              MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcD...
              -----END PUBLIC KEY-----
```

Docker Content Trust (DCT) — Legacy

Docker Content Trust is the original Docker image signing mechanism using Notary v1. It's being superseded by cosign but still found in older environments:

```
# Enable DCT
export DOCKER_CONTENT_TRUST=1
docker push myrepo/myimage:latest   # Automatically signs
docker pull myrepo/myimage:latest   # Verifies signature
```

Helm Chart Signing

Helm provides built-in chart signing and verification using GnuPG (PGP) keys. Signed charts include a .prov provenance file alongside the chart archive.

Signing Helm Charts with GPG

```
# Generate a GPG key
gpg --gen-key
# Enter: Real name, email, passphrase

# Export key for use
gpg --export-secret-keys > secring.gpg

# Package and sign the chart
helm package --sign \
  --key 'My Name <email@company.com>' \
  --keyring ./secring.gpg \
  ./mychart
# Produces: mychart-1.2.3.tgz AND mychart-1.2.3.tgz.prov

# Push both files to chart repository
# The .prov file stores the GPG signature and chart digest

# Verify a chart before installing
helm verify mychart-1.2.3.tgz \
  --keyring ./pubring.gpg

# Install with verification
helm install my-release \
  ./mychart-1.2.3.tgz \
  --verify
```

cosign for Helm Charts (Modern Approach)

cosign also supports signing Helm charts stored in OCI registries (Helm 3.8+), using the same keyless or key-based approach as container images:

```
# Push chart to OCI registry
helm push mychart-1.2.3.tgz oci://myacr.io/charts

# Sign with cosign
cosign sign oci://myacr.io/charts/mychart:1.2.3

# Verify before install
cosign verify oci://myacr.io/charts/mychart:1.2.3
```

Signing Summary

Artifact	Tool
Container images	cosign (Sigstore)
Container images (legacy)	Docker Content Trust / Notary
Helm charts (classic)	GPG / helm sign
Helm charts (OCI)	cosign
Git commits	GPG

> *Tip: Use cosign with keyless signing (OIDC) in GitHub Actions CI/CD — it eliminates key management entirely. The CI system's OIDC identity (GitHub Actions workflow) is used to sign, and the signature is logged in the public Sigstore transparency log (Rekor) for auditing.*

Q10. Describe techniques to cache Docker layers or NuGet/NPM packages in pipelines for speed optimization.

Answer

Build caching is one of the highest-ROI optimisations in CI/CD. Well-configured caching can reduce build times from 10+ minutes to 1-2 minutes. This question is also covered in the Azure document (Q6) — here we go deeper on Docker-specific techniques.

1. Docker Layer Cache — Dockerfile Instruction Order

The most fundamental optimisation: order Dockerfile instructions so that frequently changing content comes LAST. Docker caches each layer — a cache miss on any layer invalidates all subsequent layers:

```
# SLOW — package install reruns on every source code change
FROM node:20-alpine
WORKDIR /app
COPY . .                    # Changes every time → invalidates cache
RUN npm ci                  # Reruns on every code change

# FAST — package install cached unless package.json changes
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./  # Changes rarely
RUN npm ci                              # Cached unless deps change
COPY . .                               # Changes often — but layer is AFTER npm ci
RUN npm run build
```

2. BuildKit Inline Cache (Same Registry)

Use --cache-from to reuse layers from a previously built image stored in the registry. The image acts as a remote cache:

```
# Push with inline cache metadata
DOCKER_BUILDKIT=1 docker build \
  --cache-from myacr.io/myapp:cache \
  --build-arg BUILDKIT_INLINE_CACHE=1 \
  -t myacr.io/myapp:$SHA \
  -t myacr.io/myapp:cache \
  .
docker push myacr.io/myapp:cache

# Next build reuses layers from myapp:cache
# Only layers that changed are rebuilt
```

3. BuildKit Registry Cache (Separate Cache Registry)

Store the build cache separately from the production image — more control and doesn't pollute production tags:

```
DOCKER_BUILDKIT=1 docker build \
  --cache-from type=registry,ref=myacr.io/cache/myapp:main \
  --cache-to   type=registry,ref=myacr.io/cache/myapp:main,mode=max \
  -t myacr.io/myapp:$SHA .
# mode=max exports ALL layers including intermediate stages
# mode=min (default) exports only the final stage layers
```

4. GitHub Actions Cache

```
# Use docker/build-push-action with GitHub Actions cache
- name: Build and push
  uses: docker/build-push-action@v5
  with:
    push: true
    tags: myacr.io/myapp:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
# GitHub Actions stores BuildKit cache as Actions cache
# Free up to 10GB; automatically evicted when full
```

5. NPM Package Caching in GitHub Actions

```
- name: Cache NPM packages
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
```

```
      ${{ runner.os }}-npm-

- name: Install dependencies
  run: npm ci   # Uses cache if available
```

6. NuGet Package Caching (.NET)

```
- name: Cache NuGet packages
  uses: actions/cache@v4
  with:
    path: ~/.nuget/packages
    key: ${{ runner.os }}-nuget-${{ hashFiles('**/*.csproj') }}
    restore-keys: |
      ${{ runner.os }}-nuget-

- name: Restore NuGet packages
  run: dotnet restore
```

7. pip / Python Package Caching

```
- name: Cache pip packages
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('requirements*.txt') }}

- name: Install Python dependencies
  run: pip install -r requirements.txt
```

8. Self-Hosted Runners with Persistent Cache

Self-hosted runners (EC2, Azure VM, on-premise) retain Docker's build cache between runs by default — no cache push/pull needed. The Docker layer cache persists on the runner's disk:

- Benefits: Fastest possible cache (local disk), no cache upload/download overhead.
- Risks: Cache grows unbounded — implement periodic cleanup: docker builder prune --keep-storage=20GB.
- Parallel jobs on same runner share cache — can cause conflicts.

Expected Improvements

Technique	Typical Time Savings
Dockerfile layer ordering	20-40%
NPM/pip cache task	30-50%
GitHub Actions GHA cache	40-60%
Registry inline cache	40-60%
Self-hosted runners	50-75%

> *Tip: Combine Dockerfile layer ordering (free — zero infrastructure cost) with the GitHub Actions cache backend (type=gha) for 50-70% build time reduction with minimal setup. Only invest in self-hosted runners or dedicated cache registries when build frequency exceeds 20 builds/day and caching savings justify the infrastructure cost.*

Q11. Write a Dockerfile for a secure Python/Node.js API with multistage builds and best practices.

Answer

A secure, production-grade Dockerfile follows a consistent set of best practices regardless of the language: minimal base image, non-root user, no unnecessary tools, read-only filesystem where possible, verified dependencies, and proper signal handling.

Secure Python API Dockerfile

```
# syntax=docker/dockerfile:1
# ─────────────────────────────────────────────
# Stage 1: Build dependencies
# ─────────────────────────────────────────────
FROM python:3.12-slim AS builder

# Set environment variables for Python
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
```

```
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc libffi-dev libssl-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy and install Python dependencies
COPY requirements.txt ./
RUN pip install --prefix=/install --no-warn-script-location \
    -r requirements.txt

# ─────────────────────────────────────────────────
# Stage 2: Production runtime
# ─────────────────────────────────────────────────
FROM python:3.12-slim AS production

# Security: apply OS updates
RUN apt-get update && apt-get upgrade -y --no-install-recommends \
    && rm -rf /var/lib/apt/lists/*

# Security: create non-root user
RUN groupadd -g 1001 appgroup \
    && useradd -u 1001 -g appgroup -s /sbin/nologin -M appuser

# Copy installed packages from builder
COPY --from=builder /install /usr/local

WORKDIR /app

# Copy application source
COPY --chown=appuser:appgroup src/ ./src/
COPY --chown=appuser:appgroup main.py ./

# Switch to non-root user
USER appuser

# Security: read-only root filesystem
# (requires --read-only flag at runtime or in K8s securityContext)

EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')" || exit 1

# Exec form for proper signal handling
ENTRYPOINT ["python", "-m", "uvicorn"]
CMD ["main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

Secure Node.js API Dockerfile

```
# syntax=docker/dockerfile:1
# ─────────────────────────────────────────────────
# Stage 1: Install dependencies
# ─────────────────────────────────────────────────
FROM node:20.11.0-alpine3.19 AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --frozen-lockfile

# ─────────────────────────────────────────────────
# Stage 2: Build
# ─────────────────────────────────────────────────
FROM deps AS builder
COPY tsconfig.json ./
COPY src/ ./src/
RUN npm run build && npm prune --production
```

```
# ─────────────────────────────────────────────────
# Stage 3: Production
# ─────────────────────────────────────────────────
FROM node:20.11.0-alpine3.19 AS production

# Security: OS updates
RUN apk update && apk upgrade --no-cache

# Security: non-root user
RUN addgroup -g 1001 -S appgroup \
    && adduser -u 1001 -S appuser -G appgroup -H -s /sbin/nologin

WORKDIR /app

# Copy only production files
COPY --from=builder --chown=appuser:appgroup /app/dist ./dist
COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules
COPY --from=builder --chown=appuser:appgroup /app/package.json ./

# Security: read-only filesystem compatible
# Writable volumes mounted at runtime for tmp and logs

USER appuser

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health',
(r)=>process.exit(r.statusCode===200?0:1))"

ENTRYPOINT ["node"]
CMD ["dist/server.js"]
```

Security Best Practices Applied

Practice	Why It Matters
Pin exact base image version	Prevents silent image changes that introduce vulnerabilities
Multi-stage build	Build tools never ship in production image — smaller attack surface
Non-root user (UID 1001)	Root containers can escape to host — non-root limits blast radius
RUN apt-get upgrade	Apply latest OS security patches in the image
COPY --chown	Files owned by appuser — process cannot write to system directories
Exec form ENTRYPOINT/CMD	Correct SIGTERM propagation for graceful shutdown
HEALTHCHECK	Detects unhealthy containers; K8s uses this to restart/route
Read-only root filesystem	Prevents runtime file modification — malware can't persist
rm -rf /var/lib/apt/lists/*	Removes apt cache — smaller image, no package list exposure

> *Tip: After building, run docker scout cves or trivy image on the final image to verify no CVEs were introduced by the base image or dependencies. Make this a mandatory step in the CI pipeline — and fail the build if CRITICAL CVEs are found.*

Q12. How do multistage Docker builds improve security and reduce image size?

Answer

Multi-stage builds are one of Docker's most impactful features for production use. They use a single Dockerfile with multiple FROM instructions — each stage can COPY artifacts from previous stages, leaving behind everything that was only needed for the build process.

The Problem with Single-Stage Builds

A traditional Node.js Dockerfile that doesn't use multi-stage builds produces an enormous, insecure image:

```
# SINGLE-STAGE — what not to do
FROM node:20
WORKDIR /app
COPY . .
RUN npm install      # installs ALL deps including devDeps
RUN npm run build
# Final image contains:
# - Full node:20 image (1.1GB base)
# - ALL node_modules including devDependencies
# - Source code (TypeScript .ts files)
# - Build tools (tsc, jest, eslint, webpack)
# - Tests and test frameworks
# - .git directory if COPY . . was used
# Total: easily 1-2GB
```

How Multi-Stage Builds Fix Both Problems

```
# Stage 1: Build — uses full tooling
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --production

# Stage 2: Production — starts fresh with minimal base
FROM node:20-alpine AS production
# ← New FROM discards everything from Stage 1
# Only what is explicitly COPY --from=builder'd survives
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
# Final image contains: alpine base + dist/ + prod node_modules
# Total: ~150-200MB — 5-10x smaller
```

Security Improvements

- No build tools in production — Compilers (gcc, g++), transpilers (tsc, babel), bundlers (webpack, rollup), and test frameworks (jest, mocha) are only in the build stage. An attacker inside the production container cannot use these tools.
- No source code in production — TypeScript source files, JSX, test files, and documentation never ship. Intellectual property is protected and attack surface is reduced.
- No devDependencies — Development tools often have more vulnerabilities than production libraries. npm prune --production removes them before the COPY step.
- Smaller attack surface — Fewer packages = fewer CVEs. A production alpine image might have 200 packages; the full build image might have 2,000.
- Smaller image = faster security scanning — Trivy scans a 150MB image in 5 seconds vs 30 seconds for a 1.5GB image. Faster scans mean faster CI pipelines.

Size and Security Comparison — Node.js Example

Build Approach	Base Image
Single stage	node:20
Multi-stage, same base	node:20-alpine (prod)
Multi-stage, distroless	gcr.io/distroless/nodejs
Multi-stage, scratch+binary	scratch

Distroless Images — Maximum Security

Google's distroless images contain only the application runtime — no shell, no package manager, no system utilities. An attacker who gains code execution in a distroless container has almost no tools available:

```
# Stage 1: Build
FROM node:20 AS builder
```

```
WORKDIR /app
COPY . .
RUN npm ci && npm run build && npm prune --production

# Stage 2: Distroless runtime — no shell, no tools
FROM gcr.io/distroless/nodejs20-debian12 AS production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["dist/server.js"]
# Note: CMD not ENTRYPOINT — distroless node sets node as ENTRYPOINT
```

Additional Multi-Stage Patterns

- Test stage — run tests as a separate build stage; the build fails if tests fail without affecting the production image.
- Lint/security stage — run hadolint, trivy, or eslint as stages; fail early before building.

```
FROM builder AS test
RUN npm test   # Fails build if tests fail

FROM builder AS lint
RUN npx eslint src/   # Fails build if lint fails

# Production only built if test and lint pass
FROM production AS final
# ...
```

> *Tip: Use multi-stage builds by default for all production services. The only legitimate exception is for very simple scripts where build and runtime dependencies are identical (e.g., a pure shell script). For any interpreted language (Python, Ruby, PHP) or compiled language (Go, Rust, Java), multi-stage builds are always the right choice.*

Q13. Differentiate between Docker volumes and bind mounts. When do you use each in a Kubernetes environment?

Answer

Docker volumes and bind mounts both persist data beyond a container's lifecycle, but they differ in who manages the storage, where data lives, and what use cases they suit. In Kubernetes, these concepts map to PersistentVolumes and hostPath volumes respectively.

Docker Volumes — Docker-Managed Storage

Docker volumes are created and managed by Docker. The data lives inside Docker's storage directory (/var/lib/docker/volumes/ on Linux) and is completely managed by Docker — Docker decides where it physically lives. Applications reference volumes by name, not by host path.

```
# Create a named volume
docker volume create myapp-data

# Use in docker run
docker run -d \
  -v myapp-data:/var/lib/postgresql/data \
  postgres:15

# Use in docker-compose.yml
services:
  postgres:
    image: postgres:15
    volumes:
      - postgres-data:/var/lib/postgresql/data

volumes:
  postgres-data:    # Docker creates and manages this

# Inspect volume
docker volume inspect myapp-data
# Mountpoint: /var/lib/docker/volumes/myapp-data/_data

# List volumes
docker volume ls

# Remove volume (destroys data!)
```

```
docker volume rm myapp-data
```

Bind Mounts — Host-Managed Storage

Bind mounts link a specific path on the HOST filesystem directly into the container. Docker does not manage the data — it is a direct mount of a host directory or file. The host path must exist before the container starts.

```
# Bind mount syntax
docker run -d \
  -v /host/path:/container/path \
  myapp

# Examples
docker run -it \
  -v $(pwd):/app \         # Mount current directory
  -v /etc/nginx:/etc/nginx:ro \  # Mount nginx config read-only
  nginx

# docker-compose.yml
services:
  app:
    image: myapp
    volumes:
      - ./src:/app/src         # Relative path bind mount
      - /var/log/myapp:/logs   # Absolute path bind mount
```

Comparison Table

Aspect	Docker Volumes
Who manages storage	Docker (abstracted)
Host path visible	No — Docker internal
Portability	High — works across hosts
Performance	Good (optimised by Docker)
Permissions	Docker manages ownership
Best for (local)	Databases, production data
Backup/restore	docker volume backup tools
Security	Container cannot see host structure

When to Use Each in Local Development

- Use Bind Mounts for: development workflows where you want code changes reflected immediately in the container (hot reload), injecting configuration files from the host, sharing logs with the host for easier access.
- Use Volumes for: persistent database data (PostgreSQL, MySQL, MongoDB), any data that should survive container restarts and be managed independently of the host filesystem structure.

In Kubernetes — The Equivalents

Kubernetes has its own storage abstractions that map to these concepts:

Kubernetes PersistentVolume (≈ Docker Volume)

PersistentVolumes are cluster-managed storage that pods claim via PersistentVolumeClaims. The storage backend (EBS, Azure Disk, NFS) is abstracted away from the pod:

```
# PVC — pod's request for storage (like named volume)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests:
      storage: 50Gi

# Pod mounts the PVC
volumes:
- name: data
  persistentVolumeClaim:
```

```
    claimName: postgres-data
```

Kubernetes hostPath (≈ Docker Bind Mount)

hostPath mounts a specific path on the Kubernetes node into the pod. This is equivalent to Docker's bind mount:

```
volumes:
- name: host-logs
  hostPath:
    path: /var/log/myapp
    type: DirectoryOrCreate
```

> *Note: hostPath volumes in Kubernetes are considered insecure and should be avoided for most workloads. They tie pods to specific nodes (reducing scheduling flexibility) and give pods direct access to the node's filesystem (a security risk). They are acceptable only for DaemonSets that genuinely need node-level access (log collectors like Fluent Bit, metrics agents like node-exporter).*

Kubernetes-Specific Storage Types

Volume Type	Use Case
PersistentVolumeClaim (EBS/gp3)	Database storage, stateful apps
ConfigMap	Config files injected as read-only
Secret	Sensitive files (certs, passwords)
emptyDir	Temporary scratch space, sidecar sharing
hostPath	Node-level access (DaemonSets)
NFS / EFS	Shared storage across pods

> *Tip: In Kubernetes, use PersistentVolumeClaims with a StorageClass for all stateful workloads. Never use hostPath for application data — it locks your pod to a specific node and creates a security risk. Use emptyDir for temporary pod-lifetime scratch space (like a cache shared between containers in the same pod).*
